

```
=====
## What is NumPy?
=====
```

```
** NumPy stands for Numerical Python.

** NumPy is a Python library used for numerical and scientific calculations.

** NumPy is mainly used to work with arrays.

** Array means collection of values.

** NumPy arrays are faster than normal Python lists.
```

```
=====
## Why NumPy?
=====
```

NumPy is used for:

- 1) Working with large data
- 2) Mathematical calculations
- 3) Scientific calculations
- 4) Data Analysis
- 5) Machine Learning
- 6) Deep learning
- 7) Matrix operations
- 8) Statistical operations

```
=====
## NumPy Installation
=====
```

NumPy is an external library.

We need to install it using pip.

Command : pip install numpy

```
-----
import numpy as np

arr = np.array([10, 20, 30])

print(arr)
-----
```

```
=====
## What is NumPy Array?
=====
```

```
** NumPy array is a special data structure used to store multiple values.
```

```
** It is similar to list, but faster and more powerful.
```

```
** List
```

```
-- List is built-in data structures
-- List can store different data types
-- List is slower for numerical operations
-- List doesn't support direct math operations
```

```
** Numpy Array
```

```
-- NumPy array comes from NumPy library
-- Numpy array usually stores same type of data
-- Numpy array is faster
-- NumPy array supports direct mathematical operations.
```

```
-----
```

```
import numpy as np
```

```
arr = np.array([10, 20, 30])
```

```
print(arr)
print(arr * 2)
```

```
#####
```

```
list_data = [10, 20, 30]
```

```
print(list_data * 2)
```

```
-----
```

```
# ndim : ndim is used to check number of dimensions of array.
```

```
# shape : shape is used to check rows and columns of array.
```

```
# size : size is used to check total number of elements in array.
```

```
# dtype : dtype is used to check data type of array elements.
```

```
-----
```

```
import numpy as np
```

```
arr1 = np.array([10, 20, 30])
```

```
arr2 = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

```
arr3 = np.array([
    [[10, 20], [30, 40]],
    [[50, 60], [70, 80]]
])
```

```
print(arr1.ndim)
print(arr2.ndim)
```

```
print(arr3.ndim)

print(arr2.shape)
print(arr2.size)
print(arr2.dtype)

arr4 = np.array([10, 20, 30, 40, 50, 60], dtype=float)
print(arr4)

print(arr4[0])
print(arr4[-1])

print(arr2[0,0])

print(arr4[0:4])
```

```
-----

# zeros ( ) : zeros() creates an array with all zero values.

# ones ( ) : ones() creates an array with all one values.

# full ( ) : full() creates an array with same value.

# arange() : arange() creates array with range of values.
```

```
-----

arr = np.zeros(5)
print(arr)

arr = np.zeros((2,3))
print(arr)

arr = np.ones(5)
print(arr)

arr = np.ones((2,3))
print(arr)

arr = np.full(5, 10)
print(arr)

arr = np.full((2,3), 8)
print(arr)

arr = np.arange(1,11)
print(arr)

arr = np.arange(1, 11, 2)
print(arr)

arr = np.arange(1,13, 2).reshape(2,3)
print(arr)
```

```
-----

** NumPy has random module to generate random numbers.

# randint ( ) : Generate random integer numbers based on given range and size.

# rand ( ) : rand() generates random decimal values between 0 and 1 based on given size.
```

```
arr = np.random.randint(1, 9, 5)
print(arr)
```

```
arr = np.random.rand(5)
print(arr)
```

```
-----
```

reshape () : reshape() is used to change array shape.

```
arr = np.arange(1,13, 2)
arr = arr.reshape(2,3)
print(arr)
```

flatten () : flatten() converts multi-dimensional array into 1D array.

```
arr = arr.flatten()
print(arr)
```

```
=====
## NumPy Mathematical Operations
=====
```

** NumPy supports direct mathematical operations on arrays.

Ex:

```
arr = np.array([10, 20, 30])
```

```
print(arr + 5)
print(arr - 5)
print(arr * 2)
print(arr / 2)
```

** We can perform Array to Array operations like below

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
```

```
print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

```
=====
## Statistical Functions
=====
```

** NumPy provides statistical functions like:

1) sum () : sum() returns total of all values.

2) mean () : mean() returns average value.

3) median () : median() returns middle value.

4) min () : min() returns minimum value.

5) max () : max() returns maximum value.

6) std () : measures how much the data values are spread out from the mean (average).

7) var () : measures how far the values are spread from the mean, but instead of averaging the distances, it averages the squared distances.

```
-----  
classA = np.array([48, 49, 50, 51, 52]) # mean is 50  
classB = np.array([20, 40, 50, 60, 80]) # mean is 50  
  
print(np.mean(classA))  
print(np.mean(classB))  
  
print(np.std(classA))  
print(np.std(classB))
```

```
# Both classes have the same average (50).  
# Class A students performed consistently.  
# Class B students' performance varies a lot.
```

```
-----  
p1_stock_returns = [10, 11, 9, 10, 10]  
p2_stock_returns = [2, 20, -5, 25, 8]  
  
print(np.mean(p1_stock_returns))  
print(np.mean(p2_stock_returns))  
  
print(np.std(p1_stock_returns))  
print(np.std(p2_stock_returns))  
  
-----
```

**** Variance and Standard Deviation are used to measure how much data varies from its average.**

**** Standard deviation is preferred because it is in the same units as the original data, making it easier to interpret.**

**** In real-world projects, they help identify consistency, detect anomalies, measure risk, monitor quality, and prepare data for machine learning models.**

```
=====
```

```
## Boolean Indexing
```

```
=====
```

Boolean indexing is used to filter data based on condition.

Example :

```
arr = np.array([10, 20, 30, 40, 50])  
result = arr[arr > 25]  
print(result)
```

```
arr = np.array([1, 2, 3, 4, 5, 6])  
even_numbers = arr[arr % 2 == 0]  
print(even_numbers)
```

```
arr = np.array([40, 10, 30, 20])
```

```
result = np.sort(arr)
```

```
-----
```

```
# copy ( ) : copy() creates a separate copy of array.
```

Note: Changes to the copied array do not affect original array.

Ex :

```
arr1 = np.array([10, 20, 30])
```

```
arr2 = arr1.copy()
```

```
arr2[0] = 100
```

```
print(arr1)
```

```
print(arr2)
```

```
# view ( ) : view() creates a view of original array.
```

Note: Changes in view can affect original array.

```
arr1 = np.array([10, 20, 30])
```

```
arr2 = arr1.view()
```

```
arr2[0] = 300
```

```
print(arr1)
```

```
print(arr2)
```

```
=====
```

```
## Important Points
```

```
=====
```

```
** NumPy stands for Numerical Python.
```

```
** NumPy is used for numerical calculations.
```

```
** NumPy is mainly used for arrays.
```

```
** NumPy arrays are faster than Python lists.
```

```
** NumPy is an external library.
```

```
** Install NumPy using pip install numpy.
```

```
** np.array() creates NumPy array.
```

```
** ndim gives number of dimensions.
```

```
** shape gives rows and columns.
```

```
** size gives total number of elements.
```

```
** dtype gives data type of elements.
```

```
** NumPy supports indexing and slicing.
```

```
** zeros() creates array with zeros.
```

```
** ones() creates array with ones.  
** full() creates array with same value.  
** arange() creates range-based array.  
** reshape() changes array shape.  
** flatten() converts multi-dimensional array into 1D.  
** NumPy supports direct mathematical operations.  
** Boolean indexing is used for filtering.  
** copy() creates separate copy.  
** view() creates reference / view of original data.  
** NumPy is useful for Data Science, Machine Learning and AI.  
=====
```